FloraSense AI: Plant Species Identification and Crop Disease Classification Using Transfer Learning

Charul Bhanarkar
Department of Electronics and Computer Science
Ramdeobaba University
Nagpur, India

Anushka Bhongade
Department of Electronics and Computer Science
Ramdeobaba University
Nagpur, India

Anjali Balki
Department of Electronics and Computer Science
Ramdeobaba University
Nagpur, India

Arnav Domalwar
Department of Electronics and Computer Science
Ramdeobaba University
Nagpur, India

***Abstract*—Manual plant identification requires botanical expertise, and crop disease symptoms are often subtle or visually similar. This paper presents FloraSense AI, a leaf-image classification system for supported plant species identification and crop disease classification using transfer learning. Three independent classifiers are trained, each with its own label vocabulary: a 10-class Campus species model, a 12-class Kaggle species model, and a 38-class PlantVillage disease and healthy-condition model. ImageNet-pretrained MobileNetV3-Large and ResNet18 networks are fine-tuned in PyTorch on 224 × 224 RGB inputs using augmentation, cross-entropy loss, the Adam optimizer, learning-rate scheduling, and early stopping. On held-out test sets, accuracy ranges from 99.065% to 99.767%. ResNet18 records the highest accuracy on all three tasks, whereas MobileNetV3-Large has a checkpoint of about 16 MB, compared with about 43 MB for ResNet18, and lower CPU latency on every task. Inference returns top-k predictions and flags low-confidence outputs. These results are dataset-specific; potential data leakage, clean image conditions, and closed-set classes mean they should not be read as field accuracy, and independent real-world validation remains necessary.**

***Index Terms*—plant species identification, crop disease classification, transfer learning, MobileNetV3, ResNet18, deep learning**

I. INTRODUCTION

Plant identification supports education, campus biodiversity surveys, nurseries, gardens, and agricultural inspection. Manual identification depends on botanical knowledge, and visual recognition is difficult because different species can share similar leaf shapes while disease symptoms can overlap or appear subtly at early stages. Deep convolutional neural networks (CNNs) have shown strong results for image-based plant disease detection [1], and transfer learning from large pretrained models reduces the data and training effort required for a new task [2], [3].

This paper presents FloraSense AI, a leaf-image classification system that provides an initial, AI-assisted interpretation of a leaf photograph. The system comprises three independent classification tasks: Campus plant species (10 classes), Kaggle plant species (12 classes), and PlantVillage disease and healthy-condition classification (38 classes). MobileNetV3-Large is used as the main model and ResNet18 as the baseline, both initialized from ImageNet weights. The contributions are: (i) an end-to-end pipeline covering data manifests, preprocessing, augmentation, transfer learning, validation-based checkpointing, and held-out testing; (ii) a comparison of the two architectures in terms of accuracy, precision, recall, macro-F1, model size, CPU latency, and frames per second (FPS); and (iii) a confidence-aware inference stage that returns top-k candidates and flags uncertain predictions.

Section II reviews related work. Sections III and IV define the problem and the system architecture. Sections V to VII describe the datasets, methodology, and training. Section VIII reports the recorded results, and Sections IX and X discuss applications, limitations, and conclusions.

II. RELATED WORK

Mohanty *et al.* [1] trained deep CNNs on leaf images from the PlantVillage repository [4] and reported very high accuracy, while also observing markedly lower accuracy on images captured under conditions different from the training data. This domain-shift observation motivates the cautious interpretation of the results presented here.

Transfer learning reuses representations learned on a source task for a target task [2]. Networks pretrained on ImageNet [3] provide general visual features such as edges, textures, and shapes, which can be adapted to leaf images by replacing the final classification layer. ResNet [5] introduced residual connections that ease the optimization of deeper networks and is a widely used benchmark. MobileNetV3 [6] targets resource-constrained devices through efficient building blocks designed to reduce computation and memory use. FloraSense AI applies both architectures to three label spaces under the same training procedure, so that accuracy can be compared directly with model size and CPU speed.

III. PROBLEM STATEMENT AND OBJECTIVES

A. Problem Statement

Given a leaf or plant image, the system must return top-k predictions among supported species, a candidate disease or healthy-condition class for supported crops, and a confidence-aware status with an uncertainty warning where appropriate. The scope is restricted to the labels present in each training dataset; a model cannot reliably identify an arbitrary species outside its supported classes. Disease outputs are candidate visual classifications and not confirmed agricultural diagnoses.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

B. Objectives

The objectives are to: (1) build a leaf-image classification pipeline for supported species; (2) train separate models for the Campus, Kaggle species, and PlantVillage taxonomies; (3) use transfer learning and compare MobileNetV3-Large with ResNet18; (4) evaluate with accuracy, precision, recall, F1-score, confusion matrices, model size, latency, and FPS; (5) provide top-k predictions and flag low-scoring predictions as uncertain; (6) preserve class mappings, checkpoints, manifests, and environment information for reproducibility; and (7) return plant-care information only from a manually reviewed knowledge base.

IV. PROPOSED SYSTEM ARCHITECTURE

Fig. 1 shows the architecture of FloraSense AI. Image decoding and preprocessing are shared, whereas the classifiers remain separate because each dataset uses a different label vocabulary. This separation prevents class indices from being mixed and ties every output to the dataset on which the model was trained. The three models can be run on the same image, but their outputs must be interpreted independently; a disease prediction is relevant only when the input crop is compatible with the disease model's label.

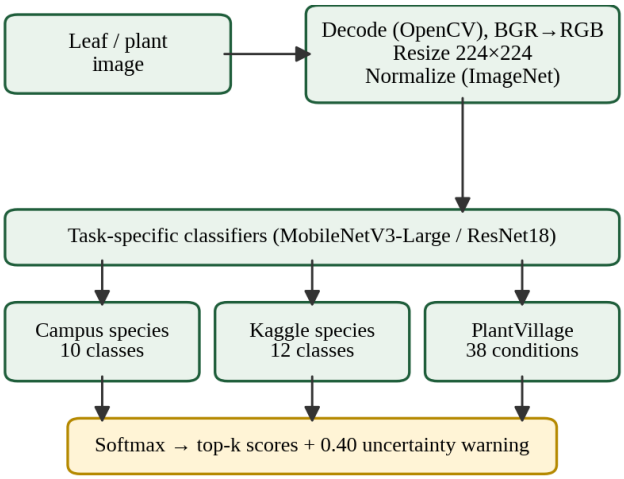


Fig. 1. End-to-end FloraSense AI architecture. Each task-specific model has its own class vocabulary.

The system is implemented as a local Jupyter Notebook in Python 3.13.1 using PyTorch and Torchvision [7] for model definition, training, and inference. OpenCV [8] performs image decoding, BGR-to-RGB conversion, and resizing, and Pillow handles image-object conversion. NumPy and Pandas support numerical operations and manifest handling, scikit-learn [9] computes classification metrics and confusion matrices, and Matplotlib and tqdm provide plots and progress display. All recorded experiments were executed on a CPU; CUDA was not used. Package versions are reproduced from the project's recorded environment metadata (PyTorch 2.14.1+cpu, Torchvision 0.29.1+cpu, OpenCV 5.0.0) and should be verified against the runtime before the environment is recreated.

An optional plant-care knowledge base holds manually entered and reviewed profiles, which may include common and scientific names, soil, sunlight, watering, temperature, common problems, management notes, source, and review date. If a recognized class has no verified profile, the system returns no care profile rather than generating advice, and it does not produce pesticide dosages, chemical treatments, medical claims, or disease-treatment instructions.

V. DATASET AND PREPROCESSING

A. Datasets and Label Spaces

Three locally stored datasets are used, summarized in Table I. The PlantVillage label space [4] contains disease categories for crops such as apple, tomato, potato, corn, grape, and pepper, together with healthy categories. The Kaggle and Campus datasets are used as supplied with the project; their detailed provenance is not specified in the source material.

TABLE I
CLASSIFICATION TASKS AND TEST SETS

Task	Classes	Test images	Purpose
Campus species	10	3,634	Campus and garden plants
Kaggle species	12	642	Indian plant species
PlantVillage	38	8,146	Crop disease and healthy conditions

Example Input Images:



B. Manifests, Splitting, and Quality Control

The pipeline creates CSV manifests recording the image path, dataset name, folder label, species label, health label, disease label, and split name, which makes data selection auditable and evaluation reproducible. For datasets without predefined splits, stratified train/validation/test splitting is used, and a 70:15:15 ratio is described in the project report; existing TRAIN, VALID, and TEST folders of the Campus dataset are preserved. Image files are discovered by supported extensions, unreadable files are rejected during OpenCV validation, class-to-index mappings are generated deterministically, and the split role of each image is recorded in the manifest.

A random image-level split can place near-duplicate images of the same physical leaf or plant in both training and test sets, which inflates test performance. Group-based splitting by plant, leaf, location, or capture session is therefore recommended for stronger evaluation (Section IX).

C. Preprocessing and Augmentation

Each image is read with OpenCV, converted from BGR to RGB, resized to 224 × 224 pixels, converted to a tensor, and normalized with ImageNet statistics. Training images are additionally augmented with random rotation, color jitter, and horizontal flipping to introduce moderate variation in orientation, brightness, and color and reduce sensitivity to capture conditions. Validation and test images use a deterministic transform without random augmentation, so that evaluation is repeatable.

VI. DEEP LEARNING METHODOLOGY

A. Multi-Class Classification with CNNs

Each task is supervised multi-class classification: every training image has one known label from a fixed vocabulary. A CNN extracts features hierarchically: early layers respond to edges, lines, and color boundaries; intermediate layers to veins, spots, margins, and textures; and deeper layers combine these into evidence that distinguishes species or disease conditions. The network outputs a logit vector **z**, and class probabilities are obtained with the softmax function

$$p_i = exp(z_i) / \Sigma_j exp(z_j) \quad (1)$$

where the class with the highest score is the top prediction.

B. Transfer Learning

Both networks start from ImageNet-pretrained weights. The final classification layer is replaced by a layer whose output size equals the number of classes of the current task (10, 12, or 38), and the model is then trained on the task data. The pretrained features offer a useful starting point instead of learning every feature from scratch.

C. MobileNetV3-Large and ResNet18

MobileNetV3-Large [6] is the main deployment candidate. Its efficient blocks, depthwise-style convolutions, inverted residual structures, and channel-attention mechanisms reduce compute and memory requirements; the recorded checkpoints are about 16 MB. ResNet18 [5] is the comparison baseline. Its residual connections add the input representation to a learned transformation,

$$y = F(x) + x \quad (2)$$

which eases optimization; its recorded checkpoints are about 43 MB.

D. Loss Function and Optimization

Training minimizes the cross-entropy loss,

$$L = - \Sigma_i y_i \log p_i \quad (3)$$

where y_i is the one-hot label, which penalizes low probability for the correct class. Parameters are updated with the Adam optimizer [10], which uses adaptive estimates of gradient moments, and weight decay is applied as regularization to help reduce overfitting.

VII. MODEL TRAINING AND IMPLEMENTATION

A. Training Configuration

Table II lists the recorded training configuration. Training and evaluation follow the workflow in Fig. 2: manifests and split records are created; Dataset and DataLoader objects are built; a pretrained architecture is loaded and its final layer replaced; and batches are passed forward to produce logits, from which the loss is computed and parameters are updated by backpropagation with Adam.

TABLE II
RECORDED TRAINING CONFIGURATION

Parameter	Recorded value
Input image size	224 × 224
Batch size	32
Learning rate	0.0001
Weight decay	0.0001
Maximum epochs	10
Early-stopping patience	4 epochs
Random seed	42
Execution device	CPU

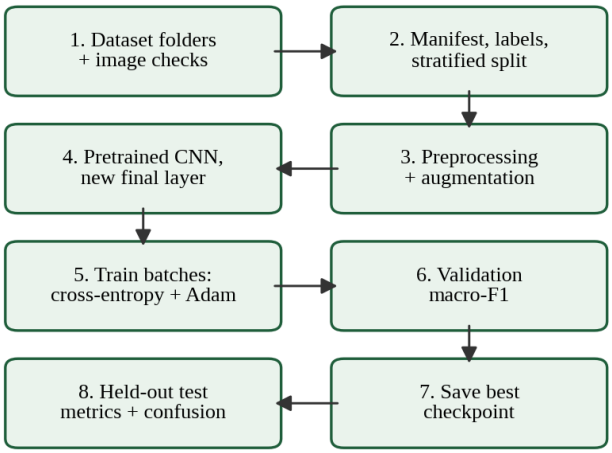


Fig. 2. Training and evaluation workflow. Test data is held out until final evaluation.

B. Validation, Checkpointing, and Early Stopping

After each epoch the model is evaluated on the validation split without weight updates. The learning rate is reduced when validation macro-F1 stops improving, the checkpoint with the best validation macro-F1 is saved, and training stops early if this metric does not improve for four epochs. The test set is not used to choose the checkpoint; the selected checkpoint is evaluated once on the held-out test split. Each checkpoint stores the architecture name, task name, model weights, class names, class-to-index mapping, image size, normalization statistics, best validation macro-F1, and random seed, so that the same inference pipeline can be reconstructed and output indices correctly translated into labels.

C. Evaluation Metrics

Performance is evaluated with accuracy, precision, recall, and F1-score, where precision is TP/(TP + FP), recall is TP/(TP + FN), and

$$F1 = 2PR / (P + R) \quad (4)$$

with macro averages computed as the unweighted mean over classes so that each class has equal influence. Confusion matrices are inspected to find class pairs that are confused, which aggregate scores can hide. Operational metrics are checkpoint size, batch-one forward-pass latency, and FPS. The latency benchmark starts from an already preprocessed 224 × 224 tensor on CPU and therefore excludes image loading and OpenCV preprocessing; end-to-end application latency will be higher.

D. Confidence-Aware Inference

At inference, the model is set to evaluation mode and the image follows the same RGB conversion, resizing, tensor conversion, and normalization used in evaluation. Logits are converted to softmax scores and the top-k candidates are returned. With a default threshold of 0.40, a top score below the threshold is reported as uncertain, and the user is advised to capture a clearer image or seek expert confirmation. Softmax scores are not automatically calibrated probabilities, so a score of 0.95 does not imply a 95% chance of being correct. A disease candidate is shown only when the predicted crop is compatible with the disease label; otherwise the output is marked unsupported or withheld.

VIII. RESULTS AND PERFORMANCE ANALYSIS

A. Classification Performance

Table III and Fig. 3 report the held-out test results from the project's final test-metrics summary. They describe saved test sets and are not an independent real-world validation.

TABLE III
HELD-OUT TEST PERFORMANCE (%)

Task (test images)	Model	Acc.	Macro P	Macro R	Macro F1
Campus (3,634)	MobileNetV3-L	99.725	99.722	99.735	99.728
Campus (3,634)	ResNet18	99.752	99.762	99.762	99.762
Kaggle (642)	MobileNetV3-L	99.065	99.088	98.926	98.993
Kaggle (642)	ResNet18	99.377	99.503	99.115	99.296
PlantVillage (8,146)	MobileNetV3-L	99.619	99.451	99.365	99.405
PlantVillage (8,146)	ResNet18	99.767	99.629	99.731	99.678

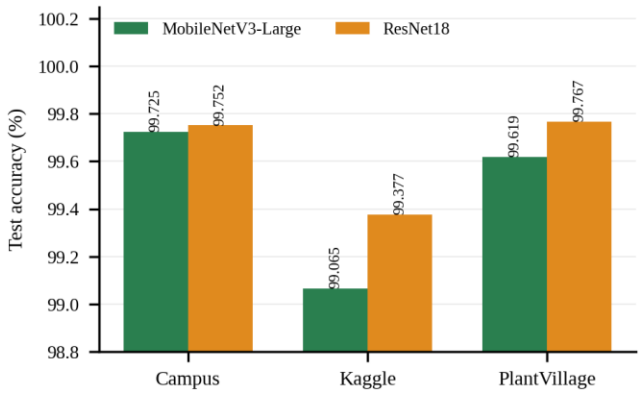


Fig. 3. Held-out test accuracy by task and architecture (y-axis truncated to 98.8–100.25%).

ResNet18 records the highest accuracy on every task, but its margin over MobileNetV3-Large is small: 0.027 percentage points (pp) for Campus, 0.312 pp for Kaggle, and 0.148 pp for PlantVillage. For the Kaggle test set of 642 images, the difference corresponds to two additional correctly classified images (638 versus 636), so it should be interpreted cautiously.

B. Per-Class Observations

For MobileNetV3-Large on the Campus task, most classes have F1 near 0.99–1.00, with Tulsi lowest at approximately 0.98. On the Kaggle task, Bael, Basil, Chinar, Jatropha, and Lemon reach F1 of 1.00, while Alstonia Scholaris is approximately 0.98. On PlantVillage, Corn Cercospora leaf spot has F1 of 0.95 with recall of 0.92, Corn Northern Leaf Blight has F1 of 0.97, and Tomato Early Blight, Spider Mites, and Target Spot are around 0.98. These harder classes merit targeted review and real-world testing.

C. Efficiency and Deployment

Table IV and Fig. 4 compare checkpoint size, CPU latency, and FPS. MobileNetV3-Large is about 16 MB per checkpoint and has lower recorded latency on all three tasks, whereas ResNet18 is about 43 MB. The ResNet18 latency on PlantVillage (69.380 ms) is much higher than its other recorded values; as these are single recorded benchmarks, repeated measurements would be needed before drawing firm conclusions about the cause. MobileNetV3-Large is therefore the recommended starting point for mobile or low-memory deployment, and ResNet18 for maximum recorded benchmark accuracy and as a research baseline.

TABLE IV
MODEL SIZE, CPU LATENCY, AND FPS

Task	Model	Size (MB)	Latency (ms)	FPS
Campus	MobileNetV3-L	16.28	12.218	81.84
Campus	ResNet18	42.73	27.883	35.86
Kaggle	MobileNetV3-L	16.29	13.424	74.49
Kaggle	ResNet18	42.74	17.555	56.96
PlantVillage	MobileNetV3-L	16.43	11.534	86.70
PlantVillage	ResNet18	42.79	69.380	14.41

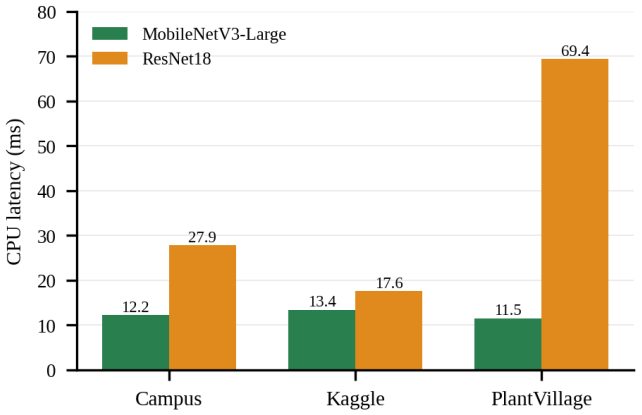


Fig. 4. Recorded CPU forward-pass latency (batch size one, preprocessed input). Lower is better.

D. Recorded Versus Real-World Performance

The recorded accuracies of 99.065%–99.767% are test-set results on the current datasets. Very high scores can occur when test images are clean, highly similar to training images, or affected by split leakage, and they are not a guarantee of field accuracy. In addition, the source material notes differences between some values in two recorded metric files, including a Kaggle ResNet18 result; the values reported here are those of the final test-metrics summary. Re-running the evaluation with the exact saved checkpoints, a single fixed manifest, verified class mappings, and recorded checkpoint hashes is recommended.

IX. APPLICATIONS, LIMITATIONS, AND FUTURE SCOPE

A. Applications

Potential uses include preliminary crop-disease screening and field inspection assistance in agriculture, plant inventory and botany education on campuses and in gardens, plant recognition in nurseries, local species cataloging and green-space surveys, and offline inference experiments on mobile and edge devices enabled by the smaller MobileNetV3-Large checkpoints.

B. Limitations

Dataset bias and domain shift: PlantVillage images are relatively clean and centered, whereas field images contain shadows, complex backgrounds, blur, insects, occlusion, multiple leaves, varied cameras, cultivars, and disease stages, so performance may decrease on such inputs. *Supported classes only:* an unsupported plant may still receive a high softmax score for a known class, and the 0.40 threshold alone does not guarantee reliable open-set rejection. *Not a diagnosis:* disease outputs are one source of visual evidence; confirmation may require expert inspection, multiple photographs, field context, or laboratory analysis. *Potential data leakage:* image-level splitting may place images of the same leaf or plant in both training and test sets, making results optimistic. *Metric inconsistency:* differences between recorded metric files remain to be resolved, as noted in Section VIII-D. *Uncalibrated confidence:* softmax scores have not been calibrated.

C. Future Scope

Priorities are independent validation on field images from different phones, lighting conditions, locations, cultivars, and disease stages; group-based splits by plant, leaf, location, or capture session; and uncertainty handling with calibration such as temperature scaling [11], reliability diagrams, class-specific thresholds, and out-of-distribution detection. Leaf detection or segmentation can help with multi-leaf or cluttered images, and Grad-CAM visualizations [12] can check whether the network attends to leaf regions rather than background cues. Further work includes automated tests for unreadable images, class mappings, checkpoint loading, and preprocessing consistency; a simple Streamlit or Gradio front end; and evaluation of TorchScript, ONNX, or quantized deployment formats after verifying model parity.

X. CONCLUSION

FloraSense AI demonstrates an end-to-end deep learning workflow for plant species identification and crop-condition classification, combining data manifests, preprocessing, augmentation, ImageNet transfer learning, validation-based checkpointing, held-out testing, efficiency benchmarking, and confidence-aware inference. On the recorded held-out test sets, ResNet18 achieves slightly higher accuracy on all three tasks (up to 99.767%), while MobileNetV3-Large offers a more practical balance of accuracy, checkpoint size (about 16 MB), and CPU latency. These figures are dataset-specific and must not be read as guaranteed field accuracy. The most important next step is independent validation on real-world images, together with calibrated uncertainty, crop-compatible disease logic, and verified care content, so that the system serves as decision support rather than a definitive diagnosis.

REFERENCES

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using deep learning for image-based plant disease detection,” *Front. Plant Sci.*, vol. 7, Art. no. 1419, 2016.

[2] S. J. Pan and Q. Yang, “A survey on transfer learning,” *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, Oct. 2010.

[3] J. Deng *et al.*, “ImageNet: A large-scale hierarchical image database,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2009, pp. 248–255.

[4] D. P. Hughes and M. Salathé, “An open access repository of images on plant health to enable the development of mobile disease diagnostics,” arXiv:1511.08060, 2015.

[5] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.

[6] A. Howard *et al.*, “Searching for MobileNetV3,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 1314–1324.

[7] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

[8] G. Bradski, “The OpenCV Library,” *Dr. Dobb’s J. Softw. Tools*, 2000.

[9] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[10] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[11] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 1321–1330.

[12] R. R. Selvaraju *et al.*, “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 618–626.